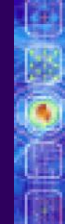




HUMAN-COMPUTER INTERACTION

THIRD
EDITION

DIX
FINLAY
ABOWD
BEALE



chapter 7

design rules

design rules

Designing for maximum usability
– the goal of interaction design

Types of Design Rules

- **Principles** of usability
 - general understanding
- **Standards** and guidelines
 - direction for design
- Design patterns (**Guidelines**)
 - capture and reuse design knowledge

Design Rules are
recommendations/principles that help
designers to create systems with
maximum usability

- Authority
 - How strongly a rule must be followed
 - High, Medium, low
- Generality
 - How broadly a rule applies across different systems, situations, or designs
 - Can I use this rule everywhere or only in a specific case?
 - If everywhere -> High generality
 - If not (only on specific case) -> Low generality

Examples:

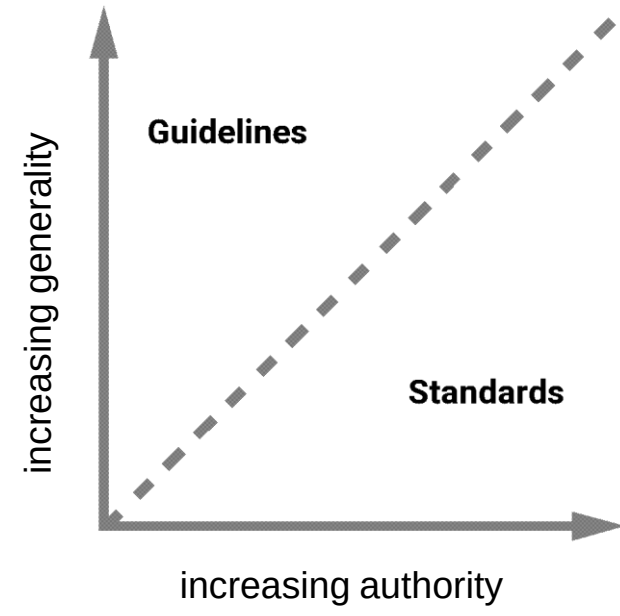
Standard -> High -> Red Light (must stop)

Guidelines -> Medium -> Speed Limits
(should follow)

Principle (Drive Safely) -> general advice

types of design rules

- principles
 - abstract design rules
 - low authority
 - high generality
- standards
 - specific design rules
 - high authority
 - limited application
- guidelines
 - lower authority
 - more general application



Design Rules	Authority		examples
Standards	High	Strict Rules	-ISO standards Accessibility Law (must follow, otherwise legal issues)
Guidelines	Medium	Recommended suggestion -> Recommended but can change Flexible rules	Use blue color for links UI Guidelines
Principle	Low	General practices	General ideas, not enforceable To make system easy to learn (incorporate Recognition rather than recall)

Principles to support usability

Learnability

the ease with which new users can begin effective interaction and achieve maximal performance

Flexibility

the multiplicity of ways the user and system exchange information

Robustness

the level of support provided the user in determining successful achievement and assessment of goal-directed behaviour

Learnability:

- How easy it is to learn the system

Flexibility:

- Different ways to interact with the system

Robustness:

- System support when things go wrong

Principles of learnability

Predictability

- determining effect of future actions based on past interaction history
- operation visibility

Synthesizability

- assessing the effect of past actions
- immediate vs. eventual honesty

Principles of learnability (ctd)

Familiarity

- how prior knowledge applies to new system
- guessability; affordance

Generalizability

- extending specific interaction knowledge to new situations

Consistency

- likeness in input/output behaviour arising from similar situations or task objectives

Why MS word is easy to learn?

- Familiar icons
- Consistent Menus
- Predictable actions

Principles of flexibility

Dialogue initiative

- freedom from system imposed constraints on input dialogue
- system vs. user pre-emptiveness

Multithreading

- ability of system to support user interaction for more than one task at a time
- concurrent vs. interleaving; multimodality

Task migratability

- passing responsibility for task execution between user and system

Principles of flexibility (ctd)

Substitutivity

- allowing equivalent values of input and output to be substituted for each other
- representation multiplicity; equal opportunity

Customizability

- modifiability of the user interface by user (adaptability) or system (adaptivity)

Substitutivity example:

List ways to open a .doc file:

- Double click on the icon
- Right click > select Open
- Open MS word > select 'open' > browse > select file
- Use keyboard > press enter
- CMD
- Open word > drag the file and drop in word

Principles of robustness

Observability

- ability of user to evaluate the internal state of the system from its perceivable representation
- browsability; defaults; reachability; persistence; operation visibility

Recoverability

- ability of user to take corrective action once an error has been recognized
- reachability; forward/backward recovery; commensurate effort

Principles of robustness (ctd)

Responsiveness

- how the user perceives the rate of communication with the system
- Stability

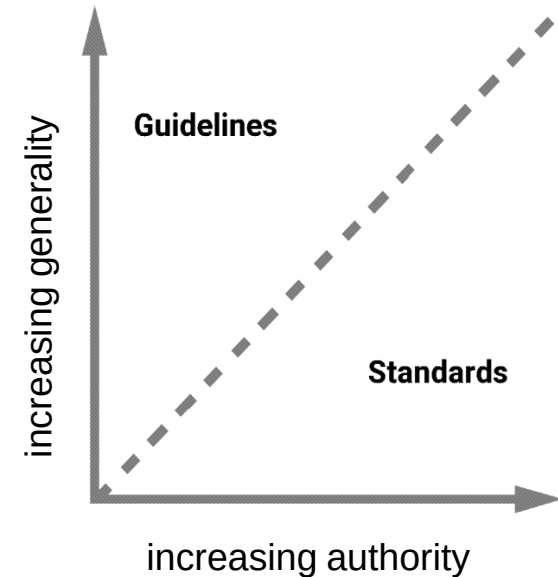
Task conformance

- degree to which system services support all of the user's tasks
- task completeness; task adequacy

Using design rules

Design rules

- suggest how to increase usability
- differ in generality and authority



Features:

- No undo
- No feedback
- Crashes frequently

Is it usable?

Standards -> IEEE, ACM, ISO

- set by national or international bodies to ensure compliance by a large community of designers standards require sound underlying theory and slowly changing technology
- hardware standards more common than software high authority and low level of detail
- ISO 9241 defines usability as effectiveness, efficiency and satisfaction with which users accomplish tasks

Why can't we just use standards for everything?

- Too Rigid
- Not adaptable for all systems

Guidelines

- more suggestive and general
- many textbooks and reports full of guidelines
- abstract guidelines (principles) applicable during early life cycle activities
- detailed guidelines (style guides) applicable during later life cycle activities
- understanding justification for guidelines aids in resolving conflicts

Golden rules and heuristics

- “Broad brush” design rules
- Useful check list for good design
- Better design using these than using nothing!
- Different collections e.g.
 - Nielsen’s 10 Heuristics (see Chapter 9)
 - Shneiderman’s 8 Golden Rules
 - Norman’s 7 Principles

Shneiderman's 8 Golden Rules

1. *Strive for consistency*
2. *Enable frequent users to use shortcuts*
3. *Offer informative feedback*
4. *Design dialogs to yield closure*
5. *Offer error prevention and simple error handling*
6. *Permit easy reversal of actions* (let the user experiment without fear)
7. *Support internal locus of control*
8. *Reduce short-term memory load*

Strive for consistency: keep everything uniform and predictable

- *User should not have to relearn the system again and again*

Offer informative feedback: System should always tell the user what is happening

- *Every action must have a reaction/feedback*

- Golden Rules ensure systems are
 - Easy
 - Safe
 - Fast
 - User friendly

Interface Design

- 1. Strive for consistency*
- 2. Enable frequent users to use shortcuts*
- 3. Offer informative feedback*

User Support

- 4. Design dialogs to yield closure*
- 5. Offer error prevention and simple error handling*

User Experience

- 6. Permit easy reversal of actions*
- 7. Support internal locus of control*
- 8. Reduce short-term memory load*

Norman's 7 Principles

- 1. Use both knowledge in the world and knowledge in the head.*
- 2. Simplify the structure of tasks.*
- 3. Make things visible: bridge the gulfs of Execution and Evaluation.*
- 4. Get the mappings right.*
- 5. Exploit the power of constraints, both natural and artificial.*
- 6. Design for error.*
- 7. When all else fails, standardize.*

1. Use both knowledge in the world and knowledge in the head.

- Knowledge in the head: what user remember*
- Knowledge in the world: what system shows*

*-> Don't force the user to remember everything.
Show information when possible*

2. Simplify the structure of tasks. -> break the task into simple and manageable steps

Online checkout: cart -> address -> payment

4. *Get the mappings right. -> controls should match their effect naturally*

- *Controls should behave the way user expects*

5. *Exploit the power of constraints, both natural and artificial. -> Limit user actions to prevent errors*

Types of constraints:

Physical (user cannot click the disabled button)

Logical (only valid options are visible)

- *Good design prevents mistakes by restricting wrong actions*

6. Design for error. -> Assume user will make mistakes -> Design to handle them

- Errors are normal -> Design should support recovery*

7. When all else fails, standardize. (Use common standards and conventions)

- Don't reinvent the wheel -> use what user already knows -> No Learning Needed*

Understanding the user

- 1. Use both knowledge in the world and knowledge in the head.*
- 2. Simplify the structure of tasks.*

Interaction Design

- 3. Make things visible: bridge the gulfs of Execution and Evaluation.*
- 4. Get the mappings right.*

Error Handling

- 5. Exploit the power of constraints, both natural and artificial.*
- 6. Design for error.*

Reusability

- 7. When all else fails, standardize.*

A design pattern is reusable solution to a common problem in a specific context

- Reusable -> can be used again and again
- Common Problem -> happens frequently
- Solution -> Already tested and works

Design patterns are proven solutions to problem designers face again and again

HCI design patterns

- An approach to reusing knowledge about successful design solutions
- Originated in architecture: Alexander
- A pattern is an invariant solution to a recurrent problem within a specific context.
- Examples
 - Light on Two Sides of Every Room (architecture)
 - Go back to a safe place (HCI)
- Patterns do not exist in isolation but are linked to other patterns in *languages* which enable complete designs to be generated

Just like architects reuse building ideas,
designer reuse interface ideas

HCI design patterns (cont.)

- Characteristics of patterns
 - capture design practice not theory (patterns come from experience, not from theory)
 - capture the essential common properties of good examples of design
 - represent design knowledge at varying levels: social, organisational, conceptual, detailed
 - embody values and can express what is humane in interface design (Embody Human Values)
 - are intuitive and readable and can therefore be used for communication between all stakeholders
 - Pattern must act as a common language among designer, developer, and client
 - a pattern language should be generative and assist in the development of complete designs.
 - Pattern language help us build entire system, not just parts

Summary

Principles for usability

- repeatable design for usability relies on maximizing benefit of one good design by abstracting out the general properties which can direct purposeful design
- The success of designing for usability requires both creative insight (new paradigms) and purposeful principled practice

Using design rules

- standards and guidelines to direct design activity